

Répartition des tâches — Équipe CyberCopilot

Projet : Assistant SOC intelligent basé sur LLM
Équipe : 4 personnes
Objectif : finaliser le projet et boucler tous les livrables avant la soutenance

Vue d'ensemble

Voici la division du travail en **4 rôles complémentaires**. Chacun a un périmètre clair, des livrables précis, un effort estimé, et une **part des 21 fichiers du code à connaître**. Personne ne se marche dessus.

Personne	Rôle	Effort	Fichiers code
1	Frontend & Design	6-10 h	5 fichiers (interface)
2	Code & Qualité	4-6 h	6 fichiers (détection)
3	Documentation & Rapport final	3-5 h	6 fichiers (stockage & sécurité)
4	Démo & Présentation	3-5 h	4 fichiers (LLM & optimisation)

Total : 21 fichiers Python répartis équitablement entre les 4 membres. Chaque personne « possède » sa part : elle doit pouvoir l'expliquer en soutenance, y ajouter ses docstrings, et répondre aux questions du jury la concernant.

Personne 1 — Frontend & Design

Mission

Produire les **maquettes visuelles** du produit, créer l'**identité graphique** et générer les **captures d'écran** pour les livrables.

Tâches

Tâche 1.1 — Maquette Figma (4-6 h)

- Suivre le brief du livrable 5 ([livrables/livrable-5-brief-figma.pdf](#))

- Créer un fichier Figma avec les 10 écrans détaillés
- Respecter la palette de couleurs et la typographie spécifiées
- Lier les écrans entre eux pour un prototype navigable

Tâche 1.2 — Captures d'écran du prototype (1 h)

- Lancer le prototype : `cd code && python app.py`
- Faire des captures de chaque écran clé :
 - Menu interactif principal
 - Détection de brute force SSH
 - Détection d'injection SQL
 - Détection de scan de ports
 - Tableau des incidents
 - Statistiques d'économie de tokens
 - Mode anonymisation activé
- Ranger dans un dossier `captures/`

Tâche 1.3 — Logo et identité (1-2 h, optionnel)

- Concevoir le logo CyberCopilot (bouclier + chevron)
- Trois variantes : couleur, monochrome, favicon
- Exporter en PNG et SVG

Fichiers code à maîtriser (5 fichiers — Interface utilisateur)

Fichier	Rôle	Ce qu'il fait
<code>app.py</code>	Point d'entrée principal	Menu interactif avec 15 options
<code>main.py</code>	Interface CLI alternative	9 commandes (analyze, list, show, chat...)
<code>src/cli.py</code>	Affichage Rich	Tableaux colorés, panels, badges sévérité
<code>src/__init__.py</code>	Module Python (vide)	Marqueur de package
<code>tests/__init__.py</code>	Module Python (vide)	Marqueur de package

À faire sur ces fichiers :

- Lire et comprendre l'enchaînement du menu (`app.py`)
- Comprendre le rôle de Rich pour l'affichage coloré (`cli.py`)
- Pouvoir expliquer la différence entre `main.py` (CLI) et `app.py` (interactif)
- **Optionnel** : ajouter quelques docstrings claires aux fonctions

Livrables attendus

- `livrable-5-maquette.fig` (fichier Figma exporté ou lien partagé)
- Dossier `captures/` avec 5 à 7 images PNG
- `logo/` avec 3 variantes (si tâche 1.3 réalisée)

Outils

- Figma (gratuit) — <https://www.figma.com>
 - GIMP ou capture d'écran système pour les captures
 - Canva si Figma trop complexe
-

Personne 2 — Code & Qualité

Mission

Améliorer la **qualité du code**, ajouter des **fonctionnalités complémentaires** et garantir la **robustesse via les tests**.

Tâches

Tâche 2.1 — Pousser le code sur GitHub (15 min)

- Créer un compte GitHub si nécessaire
- Créer un repo public `cybercopilot`
- Pousser le code local (voir commandes en bas)
- Activer GitHub Pages pour la documentation si possible

Tâche 2.2 — Enrichir les tests (2 h)

- Vérifier la couverture actuelle : `pytest --cov=src tests/`
- Ajouter des tests manquants si nécessaire
- Viser une couverture de **80 % minimum**

Tâche 2.3 — Fonctionnalités bonus (2-3 h, au choix)

- Ajouter une commande de génération de logs de test aléatoires
- Améliorer le mode chat (historique des conversations)
- Ajouter un nouveau type d'attaque (exfiltration de données)
- Ajouter une métrique de performance

Tâche 2.4 — Documentation du code (1 h)

- Ajouter des docstrings claires sur les fonctions publiques
- Générer une documentation auto avec `pdoc` ou `sphinx`

Fichiers code à maîtriser (6 fichiers — Détection des attaques)

Fichier	Rôle	Ce qu'il fait
<code>src/parser.py</code>	Parsing des logs	Extrait les événements de 4 formats (SSH, Apache, firewall, Windows)
<code>src/detector.py</code>	Règles de détection	Repère les 6 types d'attaques (brute force, SQLi, scan...)
<code>src/enricher.py</code>	Enrichissement IP	Ajoute géoloc, réputation, distinction public/privé
<code>tests/test_parser.py</code>	Tests parser	22 tests sur le parsing
<code>tests/test_detector.py</code>	Tests détecteur	16 tests sur les 6 attaques
<code>tests/test_enricher.py</code>	Tests enrichisseur	12 tests sur la géoloc

À faire sur ces fichiers :

- Comprendre comment les regex extraient les champs des logs
- Comprendre les seuils de détection (`BRUTE_FORCE_THRESHOLD` , `PORT_SCAN_THRESHOLD` , etc.)
- Pouvoir expliquer comment ajouter un 7e type d'attaque
- **Mission importante** : ajouter au moins 5 docstrings sur les fonctions de détection

Livrables attendus

- Repository GitHub public avec le code complet
- Couverture de tests ≥ 80 %
- Au moins une fonctionnalité bonus implémentée
- Documentation des modules accessible

Outils

- GitHub (compte gratuit)
- pytest, pytest-cov pour la couverture
- VSCode ou PyCharm

Mission

Compiler les 6 livrables existants en un **rapport final unique**, ajouter une **page de garde professionnelle**, et préparer la **version d'impression**.

Tâches

Tâche 3.1 — Rapport final compilé (2-3 h)

- Créer une **page de garde** avec :
 - Nom du projet : CyberCopilot
 - Sous-titre : Assistant SOC intelligent basé sur les modèles de langage
 - Logo (récupérer de Personne 1)
 - Année B2 — Promo 2026
 - Noms de l'équipe
 - Date de soutenance
- Ajouter un **sommaire général**
- Compiler les 6 livrables dans un PDF unique
- Insérer les captures d'écran fournies par Personne 1
- Ajouter une **bibliographie / sources** consultées

Tâche 3.2 — Annexes (1 h)

- Annexe A : Glossaire (SOC, SIEM, SOAR, EDR, XDR, LLM, IoC, MTDD, MTTR...)
- Annexe B : Liste des commandes du prototype
- Annexe C : Captures d'écran annotées
- Annexe D : Code source des modules critiques (extraits)

Tâche 3.3 — Mise en forme et impression (1 h)

- Uniformiser la mise en page (police, marges, en-tête, pied de page)
- Ajouter une numérotation des pages
- Vérifier les fautes d'orthographe (LanguageTool, Antidote)
- Préparer une version imprimable A4

Fichiers code à maîtriser (6 fichiers — Stockage & sécurité RGPD)

Fichier	Rôle	Ce qu'il fait
<code>src/storage.py</code>	Persistence SQLite	Base avec chmod 600, CRUD incidents, droit à l'oubli
<code>src/anonymizer.py</code>	Anonymisation RGPD	Hash IP privées, remplacement noms d'utilisateurs
<code>src/cache.py</code>	Cache LLM	Évite les appels redondants (TTL 1h)
<code>tests/test_storage.py</code>	Tests stockage	15 tests dont SQL injection safe + RGPD
<code>tests/test_anonymizer.py</code>	Tests anonymiseur	11 tests sur l'anonymisation
<code>tests/test_cache.py</code>	Tests cache	9 tests sur les clés et buckets

À faire sur ces fichiers :

- Comprendre le schéma SQLite (colonnes, index, statuts)
- Comprendre comment l'anonymisation fonctionne (regex IP, mapping users)
- Pouvoir expliquer la conformité RGPD (effacement, anonymisation, minimisation)
- **Mission importante** : ces 3 modules sont au cœur de la **sécurité du projet** (chapitre 8 du cahier des charges) — tu seras la référence pour cette partie en soutenance

Livrables attendus

- `rapport-final-cybercopilot.pdf` (50 à 80 pages)
- Glossaire complet en annexe
- Version prête à imprimer

Outils

- LaTeX (Overleaf) pour un rendu pro
- Sinon : Pandoc + Markdown
- Microsoft Word ou LibreOffice en fallback



Personne 4 — Démo & Présentation

Mission

Préparer tout ce qu'il faut pour la **soutenance orale** : slides, vidéo de démo, script du pitch.

Tâches

Tâche 4.1 — Slides PowerPoint (2 h)

Structure recommandée (15 slides max) :

1. Page de titre
2. Le problème (saturation des SOC)
3. La solution (CyberCopilot)
4. Démonstration (placeholder pour démo live)
5. Architecture en 6 modules
6. Stack technique
7. Les **6 attaques** détectées
8. Optimisation des tokens (compression + cache)
9. Sécurité (anonymisation, RGPD)
10. État du marché français
11. Concurrence et positionnement
12. Business model SaaS
13. Projections financières
14. Roadmap
15. Conclusion + Q&A

Tâche 4.2 — Vidéo de démo (1-2 h)

- Enregistrer l'écran pendant `python app.py`
- Montrer :
 - Le menu interactif
 - Une détection de chaque type d'attaque
 - Les statistiques de tokens
 - Le mode anonymisation
- Durée cible : **2 minutes maximum**
- Ajouter une voix off ou des sous-titres explicatifs

Tâche 4.3 — Script du pitch et préparation des questions (1 h)

- Rédiger un script de présentation de 10 minutes
- Préparer les réponses aux questions probables :
 - Pourquoi pas de RAG / fine-tuning ?
 - Comment vous gérez les hallucinations ?
 - Coût réel par client à grande échelle ?
 - Différenciation vs Microsoft Security Copilot ?
- Préparer un plan B si la démo plante (vidéo en backup)

Fichiers code à maîtriser (4 fichiers — LLM & optimisation tokens)

Fichier	Rôle	Ce qu'il fait
<code>src/llm_client.py</code>	Client LLM	Appels OpenAI/Anthropic + mode dégradé
<code>src/prompts.py</code>	Templates de prompts	Prompt système + format JSON strict
<code>src/compressor.py</code>	Compression intelligente	Regroupe les logs répétitifs (économie 67-94 %)
<code>tests/ test_compressor.py</code>	Tests compression	8 tests sur la normalisation

À faire sur ces fichiers :

- Comprendre comment le LLM est appelé (système + utilisateur prompts)
- Comprendre la compression (regex de normalisation, regroupement par motifs)
- Pouvoir expliquer le **mode dégradé** (fallback sans API)
- **Mission importante** : ces 4 fichiers sont le **cœur de l'argument commercial** (économie de tokens). Tu seras le porte-parole de la performance dans tes slides.

Livrables attendus

- `presentation-cybercopilot.pptx` (15 slides)
- `demo-cybercopilot.mp4` (2 minutes)
- `script-soutenance.md` avec timing et messages clés

Outils

- PowerPoint, Google Slides, ou Canva
- OBS Studio (gratuit) pour l'enregistrement écran : `sudo apt install obs-studio`
- Audacity pour la voix off
- HandBrake pour optimiser la vidéo

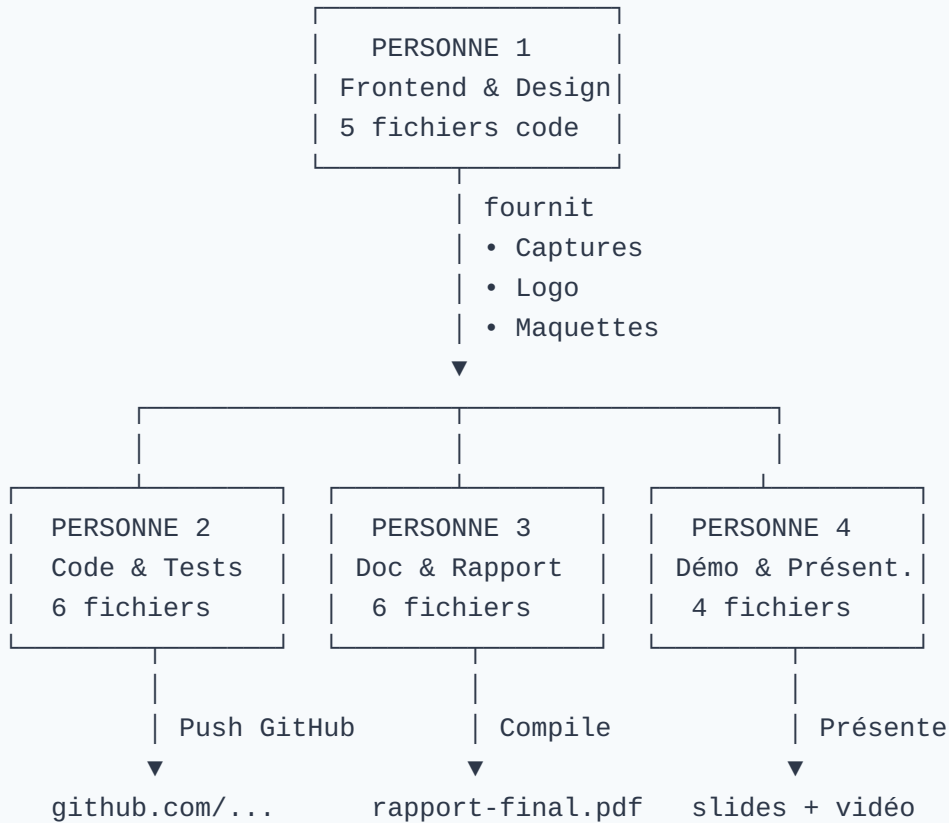


Récapitulatif global de la répartition des 21 fichiers

#	Fichier	Personne	Catégorie
1	app.py	1	Interface
2	main.py	1	Interface
3	src/__init__.py	1	Interface
4	src/cli.py	1	Interface
5	tests/__init__.py	1	Interface
6	src/parser.py	2	Détection
7	src/detector.py	2	Détection
8	src/enricher.py	2	Détection
9	tests/test_parser.py	2	Détection
10	tests/test_detector.py	2	Détection
11	tests/test_enricher.py	2	Détection
12	src/storage.py	3	Stockage
13	src/anonymizer.py	3	Sécurité
14	src/cache.py	3	Cache
15	tests/test_storage.py	3	Stockage
16	tests/test_anonymizer.py	3	Sécurité
17	tests/test_cache.py	3	Cache
18	src/llm_client.py	4	LLM
19	src/prompts.py	4	LLM
20	src/compressor.py	4	Optimisation
21	tests/test_compressor.py	4	Optimisation

Total : 5 + 6 + 6 + 4 = 21 fichiers

Coordination entre les rôles



Dépendances

- **Personne 1 livre en premier** (captures + logo) car les autres en ont besoin.
- **Personne 2 push GitHub rapidement** (15 min) pour que tout le monde travaille sur la même base.
- **Personnes 3 et 4 travaillent en parallèle** une fois les visuels disponibles.

Commandes essentielles

Pousser sur GitHub (Personne 2)

```
# 1. Sur github.com, créer un repo vide nommé "cybercopilot" (PAS d'init avec README)

# 2. Dans le terminal :
cd /home/sir30/soc-assistant
git branch -M main
git remote add origin https://github.com/TON-USERNAME/cybercopilot.git
git push -u origin main

# Si demande mot de passe : utiliser un Personal Access Token
# https://github.com/settings/tokens (case repo cochée)
```

Lancer le prototype (tout le monde)

```
cd code
pip install -r requirements.txt
cp .env.example .env # puis ajouter sa clé API
python app.py # menu interactif
```

Lancer les tests (Personne 2)

```
cd code
pytest tests/ -v
pytest tests/ --cov=src --cov-report=html # rapport de couverture
```

Générer les PDF des livrables (Personne 3)

```
cd /home/sir30/projet-soc-llm
python convert_to_pdf.py
```

Timeline suggérée (sur 1 semaine)

Jour	Personne 1	Personne 2	Personne 3	Personne 4
J1	Captures + Logo	Push GitHub + Tests	Page de garde	Plan des slides
J2	Maquettes Figma	Fonctionnalité bonus	Compilation	Création slides
J3	Finalisation visuels	Documentation code	Annexes	Vidéo de démo
J4	Lire ses 5 fichiers	Lire ses 6 fichiers	Lire ses 6 fichiers	Lire ses 4 fichiers
J5	Revue croisée entre tous les membres			
J6	Répétition générale de la soutenance			
J7	Soutenance			



Bonnes pratiques d'équipe

- **Discord ou WhatsApp** pour la communication quotidienne
- **Trello ou Notion** pour suivre l'avancement (1 colonne par personne)
- **Drive partagé** pour les fichiers volumineux (vidéo, Figma exporté)
- **GitHub Issues** pour signaler les bugs trouvés
- **Réunion de 15 min chaque soir** pour synchroniser l'avancement



Points d'attention

- **Ne jamais commiter le fichier `.env`** (clé API) — c'est dans le `.gitignore` mais vérifier
- **Sauvegarder régulièrement** le fichier Figma (export en `.fig` toutes les heures)
- **Tester la vidéo de démo** sur un autre ordinateur pour vérifier la compatibilité
- **Imprimer le rapport** 24 h avant la soutenance pour éviter les surprises

Pendant la soutenance — qui répond à quelles questions ?

Grâce à la répartition des fichiers, chaque personne devient **l'expert** de son périmètre :

Type de question du jury	Qui répond
« Comment marche l'interface ? »	Personne 1
« Comment marche la détection brute force ? »	Personne 2
« Pourquoi SQLite et pas PostgreSQL ? »	Personne 3
« Comment vous économisez des tokens ? »	Personne 4
« Quel est votre business model ? »	Personne 4
« C'est conforme au RGPD comment ? »	Personne 3
« Vous gérez quels formats de logs ? »	Personne 2
« C'est joli mais ça marche vraiment ? »	Personne 1 lance la démo

Cette répartition garantit que **personne n'est pris au dépourvu**.

Bon courage à l'équipe.